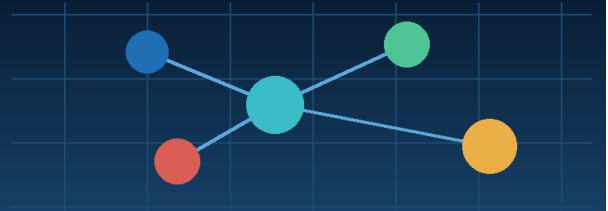


CORE PLATFORM FRAMEWORK

OFFICIAL PRODUCT DOCUMENTATION

Build · Run · Recover · Audit

master 64fd08d96392 | source 4c4248a12e69 | 186 requirements



CORE PLATFORM FRAMEWORK

기술표준서

코딩 · API · Transaction · Security · Test · Operations 표준

항목	기준
대상	개발자, Tech Lead, Reviewer, QA
정본 역할	구현·검토 표준
기준 Repository	freeangelsun/202412_01_CPF · master
기준 Commit	64fd08d963927860e8d023403dfa276931801ee5
요구사항 정보	CPF_FINAL_TARGET_REQUIREMENTS.md · 186 개
현행화 Source	4c4248a12e699c07f9f5fb11fbb33b97ca04077d

이 문서는 현재 구현의 상태 보고가 아니라, CPF 제품 사용자가 업무를 끝내기 위해 따라야 할 완성 기준 사용 계약을 중심으로 작성한다. 실제 CPF Symbol/Path 는 CURRENT SOURCE, 구현 독립 예시는 REFERENCE 로 구분한다.

문서 읽는 법과 사실 구분

표기	의미	사용 규칙
CURRENT SOURCE	최신 master 에서 직접 확인한 실제 Class/API/Route/Property/SQL/Test/경로	복사·검색 가능한 실제 이름으로 사용한다.
PRODUCT CONTRACT	CPF 가 완료된 제품으로서 제공해야 하는 사용·운영 계약	현재 구현량 때문에 축소하지 않는다.
REFERENCE	사용자가 그대로 응용할 수 있도록 제공하는 완성 기준 코드·설정·절차 예시	실제 CPF 내부 Symbol 인 것처럼 표현하지 않는다.

본문은 기능을 찾는 데서 끝나지 않는다. 적용 가능한 범위에서 선택 기준 → 준비 → 구현/설정 → 실행 → 정상 결과 → 실패/부분 실패 → 복구·대사 → 보안·감사 → Test → 운영 인계 순서로 설명한다.

1. 기본 원칙

- Owner first: 신규 API/SPI/Class 는 Core Admission Rule 을 먼저 적용하고, 하나라도 불충족이면 실제 Capability/Owner/Provider/Starter 에 둔다. provider-neutral/interface 라는 이유로 Core 에 적치하지 않는다.
- Public API/SPI/Internal 경계를 package/build/publication 에서 강제한다.
- Controller 는 orchestration/domain/persistence 를 직접 소유하지 않는다.
- Local/Remote 에서 업무 계약을 바꾸지 않는다.
- 실패를 exception 하나로 뭉개지 않고 confirmed failure 와 unknown 을 구분한다.
- 운영 위험조치는 Permission/Reason/Approval/Audit 를 코드 계약에 포함한다.

2. Package/Name 표준

● ● ● CODE · 2. Package/Name 표준

```
com.<customer>.<domain>.<feature>/  
api/  
application/  
domain/  
port/  
adapter/{persistence,remote,messaging,file}/  
config/  
validation/  
internal/
```

피할 이름	대신
CommonUtil/Helper/BaseService	역할이 드러나는 CpfDates/TransferPolicy/PaymentPort
IService	업무 의미의 Facade/Port
XXXImpl 무조건	Provider/역할 이름
Map<String,Object> public DTO	typed record/class

3. API 표준

● ● ● SOURCE · 3. API 표준

```
// GOOD
public interface AccountTransferFacade {
    TransferResult transfer(TransferRequest request);
}

// BAD: transport/framework detail 와 느슨한 Map 을 공개
public Map<String,Object> doTransfer(HttpServletRequest request);
```

4. Transaction 표준

Do	Do not
업무 일관성 요구에 따라 LOCAL/XA/OUTBOX/SAGA/TCC 선택	모든 문제를 @Transactional 하나로 해결
외부 response loss 를 UNKNOWN 으로 보존	timeout 이면 무조건 retry
same txId + attempt increment	retry 마다 transactionId 재발급
durable compensation/reconcile state	메모리 callback 만으로 분산 복구

5. SQL/Persistence 표준

- Table Owner 1 개, cross-domain direct SQL 금지
- SQL identifier 는 allowlist, value 는 binding
- stable sort + index plan + bounded paging
- optimistic version 또는 명시적 lock/fence
- Migration 에 data transform/lock/time/rollback/vendor 영향
- PII/retention classification

6. Error 표준

● ● ● JSON · 6. Error 표준

```
{
  "code": "PAY-TRANSFER-40901",
  "message": "업무 상태가 변경되어 요청을 적용할 수 없습니다.",
  "retryable": false,
  "unknownResult": false,
  "transactionId": "...",
  "details": [{"field": "amount", "reason": "RANGE"}]
}
```


7. Logging/Audit 표준

Log	Audit
기술/업무 상태, elapsed/result/error	관리자/권한/승인/재처리/download/raw access
masked payload only	before-after masked snapshot
bounded cardinality	immutable/tamper-evident
transaction/trace/segment/attempt	actor/reason/approval/result/transactionId

8. Security 표준

- input allowlist + binding; output encoding
- SSRF allowlist/canonicalization; path traversal/symlink/archive bomb 차단
- CSRF/CSP/secure cookie; credential browser 노출 금지
- Secret Source/YAML/Log 금지; provider reference 사용
- raw PII/Token/Private Key 의 audit/evidence 포함 금지
- 401(authentication)와 403(authorization)을 구분
- break-glass 는 TTL/reason/post review

9. Resilience 표준

Pattern	Required
Timeout	connect/response/overall/deadline 분리
Retry	bounded count/backoff/jitter/idempotency/replay safety
Circuit	open/half-open metric + operator visibility
Bulkhead	thread/connection/concurrency cap
Rate	global/domain/target + distributed requirement
Backpressure	bounded queue + reject/throttle
Unknown	result query/reconcile

10. Test 표준

Unit

정상 path 만 아니라 boundary/invalid/duplicate/timeout/response loss/partial/recovery 중 해당 위험을 포함한다. Test 는 실행하지 않고 성공으로 기록하지 않는다.

Contract

정상 path 만 아니라 boundary/invalid/duplicate/timeout/response loss/partial/recovery 중 해당 위험을 포함한다. Test 는 실행하지 않고 성공으로 기록하지 않는다.

Repository/Migration

정상 path 만 아니라 boundary/invalid/duplicate/timeout/response loss/partial/recovery 중 해당 위험을 포함한다. Test 는 실행하지 않고 성공으로 기록하지 않는다.

Fault Injection

정상 path 만 아니라 boundary/invalid/duplicate/timeout/response loss/partial/recovery 중 해당 위험을 포함한다. Test 는 실행하지 않고 성공으로 기록하지 않는다.

Concurrency

정상 path 만 아니라 boundary/invalid/duplicate/timeout/response loss/partial/recovery 중 해당 위험을 포함한다. Test 는 실행하지 않고 성공으로 기록하지 않는다.

Process Kill/Restart

정상 path 만 아니라 boundary/invalid/duplicate/timeout/response loss/partial/recovery 중 해당 위험을 포함한다. Test 는 실행하지 않고 성공으로 기록하지 않는다.

Multi-instance

정상 path 만 아니라 boundary/invalid/duplicate/timeout/response loss/partial/recovery 중 해당 위험을 포함한다. Test 는 실행하지 않고 성공으로 기록하지 않는다.

Security Negative Corpus

정상 path 만 아니라 boundary/invalid/duplicate/timeout/response loss/partial/recovery 중 해당 위험을 포함한다. Test 는 실행하지 않고 성공으로 기록하지 않는다.

Browser E2E

정상 path 만 아니라 boundary/invalid/duplicate/timeout/response loss/partial/recovery 중 해당 위험을 포함한다. Test 는 실행하지 않고 성공으로 기록하지 않는다.

Backup/Restore/Upgrade/Rollback

정상 path 만 아니라 boundary/invalid/duplicate/timeout/response loss/partial/recovery 중 해당 위험을 포함한다. Test 는 실행하지 않고 성공으로 기록하지 않는다.

11. Code Review Checklist

- Owner/Consumer/Public boundary
- Actual config default/fail-fast
- Transaction/unknown/idempotency/concurrency
- SQL/index/migration/rollback
- Secret/PII/masking/authorization
- Log/metric/trace/audit
- Fault/recovery test
- OpenAPI/JavaDoc/generated client
- Operations/ADM/runbook
- Dead code/stale alias/temporary artifact

Source Trace

구분	Repository 경로	확인 포인트
Developer Guide	cpf-docs/development/ DEVELOPER_GUIDE.md	development policy
Current Request	cpf-docs/work/ CPF_CURRENT_WORK_REQUEST.md	new transaction/starter/security requirements
Final Target	CPF_FINAL_TARGET_REQUIREMENTS.md	permanent completion/prohibition conditions

12. Good / Bad 구현 패턴

12.1 Remote retry

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.1 Remote retry - 금지 예

```
// [REFERENCE] BAD - timeout 을 실패로 단정하고 무조건 재호출
try { return client.call(request); }
catch (Exception e) { return client.call(request); }
```

권장 구현은 CPF 의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.1 Remote retry - 권장 예

```
// [REFERENCE] GOOD - stage/idempotency/result query 를 분리
var attempt = ledger.begin(operationId, requestHash);
try { return client.call(request, timeout); }
catch (ReadTimeoutException e) {
  return resultQuery.find(operationId)
    .orElseThrow(() -> new UnknownResultException(operationId));
}
```

GOOD	BAD
timeout stage + idempotency + bounded retry	catch(Exception){ retry(); }

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.2 Idempotency

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.2 Idempotency - 금지 예

```
// [REFERENCE] BAD - key 만 같으면 이전 결과를 반환
if (repo.exists(key)) return repo.find(key).result();
```

권장 구현은 CPF 의 Owner/Provider 경계와 operationId · transactionId · version · reason 계약을 유지합니다.

● ● ● GOOD · 12.2 Idempotency - 권장 예

```
// [REFERENCE] GOOD - scope + requestHash conflict 를 검증
var old = repo.find(scope, key);
if (old.isPresent() && !old.get().requestHash().equals(hash))
    throw new IdempotencyConflictException(key);
return old.map(IdempotencyRecord::result).orElseGet(() -> executeAndStore());
```

GOOD	BAD
key+scope+requestHash conflict	key 존재만 보고 이전 결과 반환

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.3 Transaction

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.3 Transaction - 금지 예

```
// [REFERENCE] BAD - remote 성공이 local rollback 으로 취소된다고 가정
```

```
@Transactional
```

```
void change() { local.update(); remote.commit(); throw new RuntimeException(); }
```

권장 구현은 CPF 의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.3 Transaction - 권장 예

```
// [REFERENCE] GOOD - Local 과 Remote 결과를 분리하고 보상/대사를 명시
```

```
cpfTx.execute(localDefinition, ctx -> {
```

```
    local.update(ctx.transactionId());
```

```
    outbox.enqueue(ctx.transactionId(), command);
```

```
    return null;
```

```
});
```

```
// remote 결과 유실 시 UNKNOWN_RESULT -> reconcile/compensation
```

GOOD	BAD
외부 call 과 local commit 결과 분리	remote success 가 local rollback 으로 취소된다고 가정

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.4 Lock

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.4 Lock - 금지 예

```
// [REFERENCE] BAD - owner 문자열만 저장
UPDATE lock_table SET owner_id=? WHERE lock_key=?;
```

권장 구현은 CPF의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.4 Lock - 권장 예

```
// [REFERENCE] GOOD - lease + fencing token 으로 stale writer 차단
var lease = lockProvider.acquire(lockKey, ttl);
repository.updateWithFence(id, lease.fencingToken(), change);
lease.renew();
// expired/older token write 는 DB 조건에서 0 row 로 거절
```

GOOD	BAD
lease+fencing token every protected write	DB row 에 owner 문자열만 저장

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.5 Paging

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.5 Paging - 금지 예

// [REFERENCE] BAD

```
String sql = "ORDER BY " + request.getParameter("sort");
```

권장 구현은 CPF 의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.5 Paging - 권장 예

// [REFERENCE] GOOD

```
var safeSort = CpfPersistencePolicy.requireAllowedSorts(  
    request.sorts(), Set.of("createdAt", "status", "customerNo"));  
var page = CpfPersistencePolicy.requireBoundedPage(request.page());
```

GOOD	BAD
allowlisted sort token	request sort field SQL 직접 연결

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.6 File

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.6 File - 금지 예

```
// [REFERENCE] BAD
byte[] all = Files.readAllBytes(upload);
Files.write(finalPath, all);
```

권장 구현은 CPF의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.6 File - 권장 예

```
// [REFERENCE] GOOD
try (InputStream in = source.open(); OutputStream out = temp.open()) {
    checksum = streamBounded(in, out, maxBytes);
}
verify(checksum, expectedSha256);
atomicMove(temp, finalPath);
```

GOOD	BAD
bounded stream+temp+checksum+atomic move	readAllBytes 후 final 직접 write

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server에 있는가?
- 운영자가 transactionId/operationId로 추적·복구할 수 있는가?
- Test에서 실제 fault/concurrency를 재현하는가?

12.7 Secret

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.7 Secret - 금지 예

[REFERENCE] BAD

cpf.gateway.hmac.secret: plain-text-production-secret

권장 구현은 CPF의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.7 Secret - 권장 예

[REFERENCE] GOOD

CPF_GATEWAY_HMAC_SECRET_REF=KMS:cpf/prod/gateway/hmac

application에는 provider reference/keyId 만 전달

Secret 원문은 Log/ADM/API 응답에 기록하지 않음

GOOD	BAD
provider ref/keyId	application.yml 원문 secret

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.8 Audit

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.8 Audit - 금지 예

// [REFERENCE] BAD

```
log.info("admin changed config {}", configKey);
```

권장 구현은 CPF 의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.8 Audit - 권장 예

// [REFERENCE] GOOD

```
var canonical = canonicalize(actor, action, target, beforeHash, afterHash, reason);
```

```
var record = tamperAudit.append(previousHash, canonical);
```

```
assert tamperAudit.verify(record.chainId());
```

// actor/reason/operationId/transactionId 를 함께 보존

GOOD	BAD
canonical/hash chain/actor/reason	일반 log 를 감사로그로 간주

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.9 Admin command

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.9 Admin command - 금지 예

// [REFERENCE] BAD - UI disable 을 권한으로 간주

```
<button :disabled="!canWrite" @click="dangerousAction">실행</button>
```

권장 구현은 CPF 의 Owner/Provider 경계와 operationId · transactionId · version · reason 계약을 유지합니다.

● ● ● GOOD · 12.9 Admin command - 권장 예

// [REFERENCE] GOOD - server-side permission + CAS + idempotency + reason

```
command.execute(new AdminCommand(  
    operationId, expectedVersion, idempotencyKey, reason, approvalId));
```

// 409 는 최신 상태 재조회 후 재판정; timeout 은 결과 조회 후 대사

GOOD	BAD
expectedVersion+idem+reason+approval	UI 버튼 disable 만 권한으로 간주

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.10 Unknown

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.10 Unknown - 금지 예

```
// [REFERENCE] BAD
```

```
catch (TimeoutException e) { return Result.FAILED; }
```

권장 구현은 CPF의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.10 Unknown - 권장 예

```
// [REFERENCE] GOOD
```

```
catch (TimeoutException e) {
```

```
    ledger.markUnknown(operationId, "RESPONSE_LOST");
```

```
    throw new UnknownResultException(operationId);
```

```
}
```

```
// operator: result query -> reconcile -> retry/compensate/no-op 결정
```

GOOD	BAD
durable state+result query	timeout=FAILED

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Sercret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.11 Batch restart

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.11 Batch restart - 금지 예

[REFERENCE] BAD

./gradlew runBatch --args="--job=settlement" # 매번 처음부터 재실행

권장 구현은 CPF의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.11 Batch restart - 권장 예

[REFERENCE] GOOD

동일 identifying JobParameter와 Spring Batch metadata를 확인

curl -X POST "\$ADM/api/batch/executions/\$EXECUTION_ID/retry" -H "X-CPF-Risk-Confirmed: confirmed" -d '{"reason": "checkpoint 확인 후 restart"}'

StepExecution/ExecutionContext에서 restart 지점 판정

GOOD	BAD
Spring Batch metadata+checkpoint	처음부터 무조건 재실행

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server에 있는가?
- 운영자가 transactionId/operationId로 추적·복구할 수 있는가?
- Test에서 실제 fault/concurrency를 재현하는가?

12.12 Outbox

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.12 Outbox - 금지 예

```
// [REFERENCE] BAD
@Transactional void save(){ repo.save(entity); }
publisher.publish(event); // commit 이후 best-effort
```

권장 구현은 CPF의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.12 Outbox - 권장 예

```
// [REFERENCE] GOOD
@Transactional
void save(){
    repo.save(entity);
    outbox.insert(eventId, transactionId, payload, "READY");
}
// publisher는 claim/fencing 후 publish, ACK 후 SENT; 실패는 retry/reconcile
```

GOOD	BAD
business TX와 insert atomic	commit 후 best-effort publish 만

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server에 있는가?
- 운영자가 transactionId/operationId로 추적·복구할 수 있는가?
- Test에서 실제 fault/concurrency를 재현하는가?

12.13 Frontend API

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.13 Frontend API - 금지 예

```
// [REFERENCE] BAD
const res = await fetch(`/adm/api/configs/${id}`);
if(!res.ok) alert("error");
```

권장 구현은 CPF의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.13 Frontend API - 권장 예

```
// [REFERENCE] GOOD
try {
  const result = await admConfigFindConfig({ path: { id } });
  state.value = result;
} catch (e) {
  error.value = toCpfApiError(e); // typed status/errorCode/operationId
}
```

GOOD	BAD
generated client + typed error	feature 마다 raw fetch

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.14 Download

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.14 Download - 금지 예

// [REFERENCE] BAD

```
@GetMapping("/export") byte[] exportAll(){ return repo.findAllCsv(); }
```

권장 구현은 CPF 의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.14 Download - 권장 예

// [REFERENCE] GOOD

```
requirePermission(actor, "DOWNLOAD");
requireReason(reason);
var job = exportService.create(filter, boundedLimit, expiresAt, actor);
audit.record(job.operationId(), actor, reason, filterHash);
return grantService.issueOneTimeGrant(job.id());
```

GOOD	BAD
permission/reason/limit/expiry/audit	직접 CSV endpoint 무제한

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Serret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

12.15 DB migration

잘못된 구현은 “동작 여부”가 아니라 실패 경계·중복·복구 가능성까지 포함해 판정합니다.

● ● ● BAD · 12.15 DB migration - 금지 예

-- [REFERENCE] BAD - 배포된 과거 migration 수정
-- V107__add_column.sql 내용을 직접 변경

권장 구현은 CPF의 Owner/Provider 경계와 operationId·transactionId·version·reason 계약을 유지합니다.

● ● ● GOOD · 12.15 DB migration - 권장 예

-- [REFERENCE] GOOD - immutable forward migration + verify + rollback pack
-- V112__add_order_reconcile_state.sql
ALTER TABLE cpf_order ADD reconcile_state VARCHAR(32) NOT NULL DEFAULT 'NONE';
-- verify/112_verify_order_reconcile_state.sql 로 schema/data 확인
-- rollback/R112__add_order_reconcile_state.sql 은 영향/데이터 유실 조건 명시

GOOD	BAD
new immutable version + vendor review	과거 migration 수정

Review 질문

- 정상/실패/응답 유실이 각각 어떤 상태인가?
- 재호출이 duplicate side effect 를 만들 수 있는가?
- 권한/PII/Secret/Audit 경계가 server 에 있는가?
- 운영자가 transactionId/operationId 로 추적·복구할 수 있는가?
- Test 에서 실제 fault/concurrency 를 재현하는가?

13. 기술 표준 상세 검토 매트릭스

이 절은 코드 리뷰와 설계 검토에서 기능별 규칙이 빠지지 않도록 사용하는 세부 판정표다. 각 항목은 최신 Public Contract, 실제 Consumer, 실패·복구 경로와 Test 근거를 함께 확인한다. 체크 표시만으로 판정하지 않고 Source·실행 결과·운영 조회 근거를 연결한다.

13.1 Architecture·API·Data 표준

아래 항목은 설계·구현·리뷰·인수 단계에서 적용 가능한 행을 전부 대조한다.

ID	표준 항목	필수 규칙	검증·판정
STD-A01	Module Ownership	기술 공통·고객 공통·업무·ADM/BZA·Batch·Gateway Owner 를 먼저 확정하고 역방향 의존과 타 Owner DB 직접 접근을 금지한다.	build graph 와 ArchUnit, 실제 Consumer import 를 대조한다.
STD-A02	Public API / SPI / Internal	Public API 와 SPI 에는 호환성·오류·lifecycle 계약을 두고 Internal package 는 외부 소비를 차단한다.	published artifact 소비 Test 와 package rule 이 통과해야 한다.
STD-A03	Starter 선택	Capability Profile 이 해석한 Leaf Starter 와 버전을 Manifest/Lock 에 고정하고 선택하지 않은 Provider 의존을 유입하지 않는다.	clean classpath 와 profile resolution 결과를 비교한다.
STD-A04	설정 우선순위	safe default → customer property → profile → 운영 override → per-call override 순서를 지키고 범위를 넘는 값은 거부한다.	startup fail-fast 와 runtime override audit 를 확인한다.
STD-A05	DTO 경계	API DTO, Domain model, Persistence model 을 분리하고 선택 OSS type 을 Public Contract 에 노출하지 않는다.	API signature 와 dependency graph 를 확인한다.
STD-A06	Validation	형식·범위·상호조건·권한·업무상태 검증을 분리하고 크기·개수·깊이 상한을 둔다.	negative corpus 에서 표준 오류 위치와 code 를 확인한다.
STD-A07	Paging	0-base page/size 범위와 안정 정렬을 보장하고 offset overflow, cursor 위변조·만료·version 을 검증한다.	중복·누락 0 건과 max-size 거부 Test 를 확인한다.
STD-A08	SQL Parameter	값은 binding 하고 table/column/order 같은 identifier 는 승인 allowlist 에서만 해석한다.	injection corpus 와 query plan 을 검토한다.
STD-A09	DB Ownership	Table·Migration·Runtime Query 의 Owner 를 단일화하고 Oracle/PostgreSQL/MariaDB 의미를 맞춘다.	fresh install 과 schema manifest 를 Vendor 별로 대조한다.
STD-A10	Migration	적용된 migration 은 수정하지 않고 새 version 으로 변경하며 drift 는 실패로 분류한다.	install→upgrade→rollback/forward recovery 결과를 비교한다.

ID	표준 항목	필수 규칙	검증·판정
STD-A11	Generator	중앙 template·schema metadata 를 정보으로 사용하고 생성 영역 checksum 과 사용자 영역 경계를 보존한다.	create→build/ test→remove→regenerate normalized parity 를 확인한다.
STD-A12	OpenAPI	operationId, DTO field, 권한, 오류, idempotency 와 transactionId 설명을 Runtime Contract 와 일치시킨다.	generated client drift gate 와 server route 를 대조한다.
STD-A13	JavaDoc	Public API/SPI 와 복구·동시성·보안 중요 로직에 입력·상태·오류·복구·thread-safety 를 기술한다.	javadoc publication 과 누락 gate 를 확인한다.
STD-A14	Frontend API	승인된 generated client/API boundary 를 사용하고 feature 별 raw fetch 를 만들지 않는다.	route 별 expected operationId 와 client symbol 을 대조한다.
STD-A15	다운로드	권한·사유·승인·row/size limit·만료·masking·감사를 적용하고 민감 원문 노출을 제한한다.	server authorization 과 download audit 를 확인한다.

13.2 Transaction·Messaging·Recovery 표준

아래 항목은 설계·구현·리뷰·인수 단계에서 적용 가능한 행을 전부 대조한다.

ID	표준 항목	필수 규칙	검증·판정
STD-T01	LOCAL	단일 Resource 변경은 명시된 local transaction boundary 에서 commit/rollback 하고 외부 side effect 를 긴 DB transaction 에 묶지 않는다.	commit 전·후 fault injection 과 rollback 결과를 확인한다.
STD-T02	XA/JTA	강한 원자성이 필요한 DB+DB 또는 DB+JMS 에서만 선택하고 Provider-neutral contract 와 XA resource enlistment 를 유지한다.	prepare/commit/rollback 및 timeout 결과를 확인한다.
STD-T03	XA Recovery	prepare 완료 뒤 process kill 시 recovery log 와 startup scan 으로 in-doubt 를 판정하며 중복 recovery writer 를 fencing 한다.	kill→restart→scan→최종 판정 Harness 를 실행한다.
STD-T04	Outbox	업무 변경과 Outbox INSERT 를 동일 local transaction 으로 묶고 stable eventId 와 claim lease/fencing 을 사용한다.	ACK loss·publisher kill 뒤 duplicate business effect 0 건을 확인한다.
STD-T05	Inbox/Dedup	message ID 와 consumer identity 기준으로 concurrent duplicate 를 한 번만 처리하며 processing/replay/retention 상태를 남긴다.	consumer kill/restart 와 동시 duplicate Test 를 확인한다.
STD-T06	Saga	장기 A→B→C 흐름은 단계 상태와 역순 compensation 을 기록하고 compensation 실패를 별도 retry/UNKNOWN 으로 남긴다.	C 실패·보상 실패·process kill 시 나리오를 검증한다.

ID	표준 항목	필수 규칙	검증·판정
STD-T07	TCC	Try/Confirm/Cancel 을 각각 역등하게 만들고 empty rollback·hanging·duplicate confirm/cancel 을 구분한다.	timeout/kill/recovery 와 manual review 전환을 확인한다.
STD-T08	Idempotency	canonical request hash·scope·TTL·terminal result replay 를 사용하고 같은 key 의 다른 payload 는 conflict 로 거부한다.	동시 요청 race 와 hash mismatch Test 를 확인한다.
STD-T09	Lock/Lease/Fencing	owner·request·epoch·version·lease 와 단조 증가 fencing token 을 함께 검증하여 stale writer 를 차단한다.	lease 만료·takeover·늦은 write Test 를 실행한다.
STD-T10	Deadline	전체 deadline budget 을 하위 DB/HTTP/Broker/File 호출에 배분하고 남은 시간을 넘는 새 시도를 시작하지 않는다.	timeout budget 소진과 cancel/cleanup 상태를 확인한다.
STD-T11	Retry	역등 또는 결과 조회가 가능한 operation 에만 제한된 retry 를 적용하고 backoff/jitter 와 storm 방지를 둔다.	retry count cap 과 dependency outage Test 를 확인한다.
STD-T12	UNKNOWN_RESULT	외부 side effect 가능 상태에서 응답이 유실되면 성공·실패로 단정하지 않고 operation/상대 상태를 조회한 뒤 reconcile 한다.	response loss 후 상태 조회·대사·수동 확정 경로를 확인한다.
STD-T13	Message Envelope	messageId, transactionId, schema version, key/partition, TTL, producer/environment binding 과 size 제한을 유지한다.	producer/consumer compatibility 와 malformed message 거부를 확인한다.
STD-T14	DLQ Replay	원인 수정·대상 범위·순서·idempotency·승인 후 replay 하고 원 거래와 새 attempt 를 연결한다.	poison 격리와 부분 replay 결과를 대조한다.
STD-T15	Batch Restart	Spring Batch metadata/checkpoint 를 기준으로 restart 하고 CPF execution/lease/fencing/UNKNOWN 원장과 ID 를 연결한다.	process kill 이후 checkpoint 재개와 duplicate item 0 건을 확인한다.

13.3 Security·Audit·Operations 표준

아래 항목은 설계·구현·리뷰·인수 단계에서 적용 가능한 행을 전부 대조한다.

ID	표준 항목	필수 규칙	검증·판정
STD-S01	OAuth2/JWT	issuer·audience·expiry·scope/role mapping 을 검증하고 current principal/tenant 접근은 공통 Security Context 를 사용한다.	401/403, 만료·잘못된 audience·claim mapping Test 를 확인한다.

ID	표준 항목	필수 규칙	검증·판정
STD-S02	OIDC SSO	외부 IdP login/logout/session/token 만료·갱신과 user/tenant/group mapping 을 BFF/session 경계와 연결한다.	IdP 장애·logout·session revoke 흐름을 확인한다.
STD-S03	Session/BFF	Browser storage·URL·DOM·console 에 access/refresh token 과 session ID 원문을 남기지 않고 fixation·rotation·concurrency 를 통제한다.	credential leak scan 과 강제 logout Test 를 확인한다.
STD-S04	Secret Provider	Secret 원문을 Source/YAML/Log/ADM/Evidence 에 기록하지 않고 provider health·rotation·revocation·timeout 을 다룬다.	provider failure 와 rotation 이후 consumer 재연결을 확인한다.
STD-S05	KMS/HSM	keyId/keyVersion 으로 외부 KMS/HSM 또는 PKCS#11 provider 를 선택하고 private key 원문을 애플리케이션에 노출하지 않는다.	sign/verify·provider timeout·revocation Test 를 확인한다.
STD-S06	Digital Signature	algorithm, keyId/version, certificate, signature metadata 를 함께 기록하고 잘못된 signature·만료·revocation 을 거부한다.	negative signature corpus 와 audit 를 확인한다.
STD-S07	PII Masking	field classification 과 context-aware masking 을 응답·로그·다운로드·Evidence 에 적용하고 raw 조치는 별도 권한·사유·승인을 요구한다.	PII/credential leak scan 을 실행한다.
STD-S08	Tamper-evident Audit	canonical payload 와 previousHash/currentHash chain 을 append-only 로 기록하고 선택적 signature 로 무결성을 검증한다.	수정·삭제·순서 변경 탐지 Test 를 확인한다.
STD-S09	Admin Command	위험 조치는 requester/approver 분리, reason, expected version, immutable command hash, expiry 와 audit 를 요구한다.	자기승인·stale version·만료 승인 거부를 확인한다.
STD-S10	Break-glass	별도 권한·TTL·긴급 사유·사용 범위·사후 Review 를 남기고 일반 권한 경로와 구분한다.	세션 만료와 review closure audit 를 확인한다.
STD-S11	Observability	transactionId·trace/span·segment·attempt·instance/job/execution 을 구조화하고 cardinality 와 masking 을 제한한다.	로그·Metric·Trace 에서 동일 거래 계보를 대조한다.

ID	표준 항목	필수 규칙	검증·판정
STD-S12	ADM Timeline	transactionId 하나로 Local/Remote/Event/Batch/Gateway/File 구간을 연결하며 query 실패는 partial/query-failed 로 구분한다.	source 별 누락·지연·조희 실패 표시를 확인한다.
STD-S13	Runtime Config	변경 전 preview 와 target/version 을 확정하고 부분 적용·응답 유실은 operation 상태를 조희해 reconcile 한다.	expected version conflict 와 selective rollback 을 확인한다.
STD-S14	Artifact Trust	release artifact 의 manifest·SHA-256·signature·build SHA·환경 binding 을 검증하고 불일치 artifact 를 활성화하지 않는다.	tamper·wrong environment·wrong version 거부 Test 를 확인한다.
STD-S15	Evidence	실행 명령, exact source SHA, 환경/topology, 시작·종료, exit code, 결과와 artifact hash 를 남기고 실행하지 않은 Test 를 성공으로 기록하지 않는다.	Evidence 가 현재 Commit 과 입력 artifact 를 가리키는지 역추적한다.

13.4 판정 기록 원칙

- 검토자는 해당 항목의 Requirement ID 또는 변경 요청과 Source Path, 실제 Consumer, Test/Harness, 운영 확인 위치를 같은 Review Record 에 연결한다.
- 문서량 증가만으로 품질을 판정하지 않는다. 다만 기존 정보보다 설명·판정 단위가 줄어드는 변경은 삭제·통합 근거와 대체 위치가 명확하지 않으면 재검토한다.
- 규칙을 통합해 행 수가 줄어드는 경우에도 삭제된 규칙의 의미가 새 Owner 절에 남아 있는지 양방향으로 확인한다.

13.5 Review Record 필수 필드

표준 위반 여부를 판정할 때 아래 필드를 같은 Review Record 에 남긴다.

필드	기록 내용	판정 기준
Requirement / Change	Requirement ID, 변경 목적, 적용 범위	검토 대상이 무엇인지 단일하게 식별된다.
Source / Consumer	exact SHA, Source Path, Symbol, 실제 Consumer	정본 Source 와 실제 호출 경로가 연결된다.
Contract / State	입력·출력·상태·오류·idempotency·timeout	정상과 오류·결과 불명 경계가 구분된다.
Failure / Recovery	fault 지점, retry/restart/reconcile/compensation/rollback	실패 후 정상화 방법과 종료 판정이 기록된다.
Security / Audit	권한, masking, reason, approval, audit	위험행위와 민감정보 경계가 server 에서 확인된다.
Test / Operations	Test/Harness 명령, 결과, ADM/Log/Metric/Trace 확인	실행 근거와 운영 확인 위치가 현재 Source 에 연결된다.
Judgement	완료/부분 구현/미구현/미검증/실패/재확인 필요	판정 근거와 남은 검증 범위가 분리된다.

14. 현행 Source 기반 규칙 보강

Rule	표준	Current Source
TX-API	CpfTransactionOperations 를 업무 transaction 진입점으로 사용하고 Provider API 직접 결합을 제한한다.	cpf-core/api/transaction
TX-DEFAULT	LOCAL 기본 정의는 JTA Provider 없이 생성·실행 가능해야 한다.	CpfTransactionDefinition.localDefault()
PERSIST	공통 CRUD/Search 는 선택적으로 Public Port 를 사용하고 복잡 업무 Query 는 Domain Port/native escape hatch 로 분리한다.	CpfCrudRepositoryPort
PERSIST-GUARD	sort/filter/page/bulk 상한은 CpfPersistencePolicy 를 공통 경계로 사용한다.	CpfPersistencePolicy
JPA	JPA convenience API 는 timeout·exception translation·transactionId observation·audit hook 을 함께 적용한다.	CpfJpaOperations
AI	AI 요청은 transactionId/model/prompt/risk/timeout/humanApproved 를 포함하고 Provider routing 은 Starter 가 담당한다.	CpfAiRequest/CpfAiOperations
OIDC	claim name 과 role/scope prefix 를 typed property 로 관리하고 safeClaimNames 외 원문 claim 노출을 제한한다.	CpfOidcProperties
SIGN	signature API 는 transactionId/keyId/algorithm/canonical payload 를 명시한다.	CpfDigitalSignatureOperations
TAMPER	감사 무결성은 append/hash-chain store contract 로 분리하고 JDBC Provider 는 그 계약을 구현한다.	CpfTamperAuditStore
SOAP	SOAP 공통 설정은 connect/read timeout 을 제공하고 endpoint/action 의미는 호출 Consumer 가 명시한다.	CpfSoapProperties
SEC-CONTEXT	Resource Server 업무 코드는 raw claim 접근 보다 typed Security Context 를 우선하고 issuer/audience/expiry 와 role/scope mapping 경계를 유지한다.	CpfSecurityContext / CpfResourceServerProperties
XA-RECOVERY	XA prepare 이후 in-doubt 복구는 startup recovery scan 과 recovery resource 등록을 통해 수행하고 중복 recovery 를 식별 가능하게 남긴다.	CpfNarayanaRecoveryRegistrar / CpfXaRecoveryResourceProvider
XA-DATASOURCE	XA Resource 조립은 Vendor XADataSource factory 경계를 사용하고 일반 LOCAL DataSource 와 선택 경로를 분리한다.	CpfXaDataSourceFactory
TCC-PARTICIPANT	TCC Try/Confirm/Cancel 은 participant 계 약으로 분리하고 각 phase 의 멍등성·empty rollback·hanging 처리를 업무 Consumer 가 명시한다.	CpfTccParticipant / CpfTccPhase

Rule	표준	Current Source
TCC-RECOVERY	TCC timeout/process-loss 후에는 recovery operation 으로 상태를 조화·대사하고 UNKNOWN 을 성공/실패로 임의 확정하지 않는다.	CpfTccRecoveryOperations / CpfTccResult
INBOX-DEDUP	메시지 수신 side effect 전 Inbox/Dedup 원장을 사용하고 concurrent duplicate 와 replay 에서 업무 부작용 중복을 차단한다.	CpfBrokerInboxPort
SAGA-UNKNOWN	Saga compensation 자체의 결과 불명은 별도 UNKNOWN 상태와 예외로 보존하고 manual recovery/reconcile 흐름과 연결한다.	CpfSagaStatus / CpfSagaUnknownOutcomeException
CONFIG-CATALOG	Property 는 typed descriptor/catalog 에 등록해 mutability, restart requirement, secret separation 을 Consumer 와 운영자가 동일 계약으로 해석한다.	CpfConfigCatalog / CpfConfigDescriptor / CpfConfigPolicy
SORT-POLICY	정렬 field 와 column 수는 공통 allowlist 정책으로 검증하고 요청 문자열을 SQL/JPQL 정렬식에 직접 결합하지 않는다.	CpfSortPolicy
LOCK-REPOSITORY	영속 조회의 lock mode 는 typed contract 로 전달하고 업무 상태/동시성 정책 없이 Provider lock hint 를 임의 사용하지 않는다.	CpfLockingRepositoryPort / CpfLockMode
BULK-BOUND	Bulk create/update/delete 는 최대 건수와 flush 경계를 제한하고 무제한 Collection 전체 적재를 기본 경로로 사용하지 않는다.	CpfBulkRepositoryPort / CpfPersistencePolicy
AUDIT-JDBC	Tamper-evident audit 의 append/head/hash-chain 영속화는 Audit Store 계약과 JDBC Adapter 를 분리하고 일반 application log 로 대체하지 않는다.	CpfTamperAuditStore / JdbcCpfTamperAuditStore
KEY-MGMT	Key 조회·rotation·revocation·health 는 Key Management 계약으로 수행하고 Private Key 원문을 Consumer/로그/운영 응답에 반환하지 않는다.	CpfKeyManagementOperations / CpfKeyLifecycleProvider

16. 실전 Do / Don't 코드 표준

16.1 Transaction

● ● ● CODE · CPF Transaction Contract 사용

```
return tx.execute(  
    new CpfTransactionDefinition(LOCAL, REQUIRED, DEFAULT, Duration.ofSeconds(3), false),  
    context -> service.execute(command, context));
```

● ● ● SOURCE · Controller 에서 수동 transaction/retry 조립

```
@PostMapping  
void bad() {  
    for (int i=0; i<10; i++) {  
        try { jdbc.update(...); remote.call(); return; }  
        catch (Exception ignored) {}  
    }  
}
```

16.2 Persistence

● ● ● CODE · sort/filter allowlist

```
CpfPersistencePolicy.requireAllowedSorts(request.sorts(), ALLOWED_SORT_FIELDS);  
CpfPersistencePolicy.requireAllowedFilters(request.search(), ALLOWED_FILTER_FIELDS);
```

● ● ● SQL · user input SQL concatenation

```
String sql = "SELECT * FROM pay_payment ORDER BY " + request.sortField(); // 금지
```

16.3 Recovery

● ● ● CODE · DO

timeout → identify side-effect boundary → result query → reconcile → retry only when proven safe

● ● ● CODE · DON'T

timeout → immediate blind retry → duplicate side effect

문서 기준 및 검증 범위

항목	기준
Repository / Branch	freeangelsun/202412_01_CPF / master
Work start / latest reviewed HEAD	64fd08d963927860e8d023403dfa276931801ee5 (07_17)
Latest product source change basis	4c4248a12e699c07f9f5fb11fbb33b97ca04077d (07_16 canonical currentization source)
Canonical Requirement	186 개
CURRENT SOURCE	실제 Repository 에서 확인한 Symbol/경로
PRODUCT CONTRACT	현재 구현량과 분리한 제품 목표 계약
REFERENCE	복사·응용 가능한 완성 기준 예시; CURRENT SOURCE 아님
Runtime qualification	문서 세션에서 직접 실행하지 않은 DB/Browser/Multi-instance/Fault 는 미검증

17. Core Admission·Capability Ownership 표준

금지	대신 해야 할 일
Provider-neutral interface 라는 이유로 cpf-core 배치	전용 Consumer/Owner 를 찾고 Capability Contract 로 배치
Optional 기능 API 를 Core 에 상시 의존	Optional Capability + 미사용 0-footprint
Logging runtime 을 Core 에서 조립	platform-operations observability/logging capability
Fixed-Length Layout/Parser 를 Core 에 배치	fixedlength-core, Spring Runtime 은 fixedlength
모든 편의 API 를 Core 로 승격	Error/Transaction/Context 만 전역 Kernel, 나머지는 Owner 별 Public API

● ● ● CODE · BAD · Core dumping

```
// BAD
package com.cpf.core.api.file;
public interface AttachmentScanner { /* File 전용 계약 */ }

package com.cpf.core.api.ai;
public interface ModelRouter { /* AI 전용 계약 */ }
```

● ● ● CODE · GOOD · Owner-first

```
// GOOD (개념 예시)
// global semantics -> cpf-core
TransactionContext currentTransaction();

// dedicated contracts -> File / AI / Batch / Gateway Capability Owner
// provider implementation -> selected Starter / Provider
// 실제 package 이름은 CURRENT SOURCE 와 PRODUCT CONTRACT 상태를 구분한다.
```

검수 질문	PASS 기준
Optional 기능을 제거해도 Core 가 독립적인가?	예
전용 Owner 없이 대부분 Capability 가 의미를 공유하는가?	예
기술 Provider 를 바꿔도 의미가 유지되는가?	예
전용 DTO/Port 가 Core 에 섞이지 않았는가?	예
CURRENT SOURCE gap 을 목표 구조처럼 설명하지 않았는가?	예